GameBox

Multiplayer Integration Specification

Game Launch, Session, Matchmaking, Encryption & Score Submission Guide

Document Title	GameBox Multiplayer Integration Specification
Prepared For	Game Vendor / Development Partner
Prepared By	GameBox Platform Engineering Team
Version	1.3
Environment	Staging / Test (`ui-gamebox.stacknize.com`)
Classification	Confidential — Vendor Integration Use Only
Status	Ready for QA Testing & Client Validation

Table of Contents

1. Overview 4
2. Integration Flow Summary 4
3. Game Launch 4
3.1 Launch URL Format 4
3.2 Reading Launch Data 4
4. Launch Token Decryption 4
4.1 Decryption Key 4
4.2 Decryption Process 4
4.3 Expected Response After Decryption 5
4.4 Key Fields Reference 5
4.5 Token Implementation 6
5. Nakama Connection & Session Setup 9
5.1 Connection Details 9
5.2 Authentication Flow 9
5.3 Nakama Authentication Details 9
5.4 Session Creation Process 10
5.5 Socket Connection Creation 10
5.6 Reconnection Handling 10
6. Matchmaking 11
6.1 Matchmaking Request 11
6.2 Matchmaking Response 11
6.3 Bot Fill Timing 11
6.4 Error Handling 12
6.5 SDK & Implementation Reference 12
6.6 Match Cancellation Flow 12
7. Bot Player Details & Session Fields 13
7.1 Bot Identification 13
7.2 Dynamic Bot Profile Selection 13
7.3 Bot Player Metadata Payload 13
7.4 In-Game Session Context 13
8. Match Connection 13
8.1 Connecting to the Game Server 13
8.2 Match Found Event Payload 14
8.3 Dedicated Server Authentication 14
8.4 Reconnect Flow 15
9. Gameplay Communication 15
9.1 Sending Player Input 15
9.2 Receiving Game State Updates 16
10. Match End 16
10.1 Opponent Disconnect Flow 16
10.2 Normal Match Completion 16
10.3 Match End Event Payload 17
10.4 Client Reaction on Receipt 17
11. Server Error Events 17
11.1 Invalid Token / Unauthorized 17
11.2 Room Closed 17
11.3 Match Expired 18

11.4 Server Unavailable 18
12. Score Encryption 18
12.1 Sample Score Payload 18
12.2 Encryption Process 18
12.3 Token Structure 19
13. Algorithms Used 19
14. Compatibility Requirement 19
15. Score Submission Flow 19
16. Multiplayer Game Score Submission API 19
16.1 Endpoint 19
16.2 Request Payload 20
16.3 cURL Example 20
16.4 Response 20
16.5 Functionality 20

1. Overview

This document defines the integration requirements for game developers connecting multiplayer titles to the GameBox platform. It covers how to read game launch data, decrypt the launch token, connect to and authenticate against the Nakama backend, run matchmaking, handle bot fallback, manage in-match connectivity, encrypt score data, and submit results through the multiplayer score submission API.

Vendors must implement the exact algorithms, token structure, compression format, and encoding scheme described below to ensure compatibility with the GameBox platform.

2. Integration Flow Summary

At a high level, integration consists of six stages:

1. Game launch — the platform redirects the user to the game URL with an encrypted launch token.
2. Launch data decryption — the game decrypts the token to retrieve user and game context.
3. Nakama connection & authentication — the game connects to the Nakama backend and establishes a session.
4. Matchmaking — the game queues the player and receives either a human opponent or a bot fallback.
5. Gameplay & score encryption — after the match, the game encrypts the score payload using the platform-provided key.
6. Score submission — the encrypted payload is submitted to the GameBox Multiplayer Score Submission API.

3. Game Launch

3.1 Launch URL Format

When a game is launched, the platform redirects the user to the game URL and appends an encrypted query parameter named encryptedString.

Example launch URL:

```
https://ui-gamebox.stacknize.com/?encryptedString=eyJ2IjoxLCJjdCI6IjhScXhUNUd5Z1ZS...
```

3.2 Reading Launch Data

The game client reads the value of the encryptedString query parameter from the browser URL. This value contains compressed and encrypted game launch information that must be decrypted before use.

4. Launch Token Decryption

4.1 Decryption Key

A shared decryption key is used to decrypt the launch token (test/reference value shown below):

```
GameDataDecryptionKey = 'J9k8L7m6N5p4Q3r2'
```

4.2 Decryption Process

Decrypt the launch token using the following sequence:

1. Read the encryptedString value.
2. Validate encryptedString and the decryption key.
3. Base64URL-decode encryptedString.
4. Parse the resulting token JSON.
5. Validate that the token contains v, ct, and mac fields.
6. Base64URL-decode ct (the ciphertext).
7. Base64URL-decode mac.
8. Generate an HMAC-SHA256 value using createMac().
9. Compare the generated MAC with the received MAC.
10. Generate the AES key using getKeyBuffer(decryptionKey).
11. Generate the IV using getIv(decryptionKey).
12. Decrypt the ciphertext using AES-128-CBC.
13. Decompress the decrypted data using zlib InflateRaw.
14. Convert the decompressed data to a UTF-8 string.
15. Parse the resulting payload as JSON.

4.3 Expected Response After Decryption


```
{
  "v1": true,
  "data": {
    "userId": "1878931",
    "display_name": "User",
    "username": "user_1878931",
    "isbot": false,
    "user_skill": "e",
    "user_level": 123,
    "avatar_url": "https://ui-gamebox.stacknize.com/assets/img/user.webp",
    "gameId": "181",
    "gamethumbnailurl": "assets/all_images/BallPhysicGame/BallPhysicGame.webp",
    "canPlay": true,
    "session_id": "play-and-win-270-1787666045680-tj345phc",
    "bearerToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "game-category-slug": "ball-physic-game",
    "gameScoreAPIUrl": "https://ui-gamebox.stacknize.com/submitMultiplayerScore",
    "checkCreditScoreAPIUrl": "https://api- gamebox.stacknize.com/api/v1/user/checkCreditGameAccess",
    "redirectUrl": "https://ui-gamebox.stacknize.com/",
    "nakamaURl": "https://nakama.stacknize.com",
    "matchmakingUrl": "https://multiplayer.stacknize.com",
    "scoreEncryptionKey": "S1c2O3r4E5n6C7k8"
  }
}
```


Note: Use the redirectUrl value received in the decrypted payload when canPlay is false. Redirect the user to this URL instead of allowing game launch or gameplay actions.

Example:


```
{
  "redirectUrl": "https://ui-gamebox.stacknize.com/"
}
```


4.4 Key Fields Reference

Field	Description
`gameId`	Unique identifier for the game.

Field	Description
`userId`	Identifier of the currently logged-in user.
`userName`	Display name of the user.
`canPlay`	Boolean flag indicating whether the game launch is permitted.
`gameScoreAPIUrl`	Endpoint to call when submitting the multiplayer score.
`scoreEncryptionKey`	Key that must be used to encrypt the score payload before submission.
`nakamaURl`	Use the `nakamaURl` value received in the decrypted payload for establishing Nakama client/server connections and related multiplayer operations.
`matchmakingUrl`	Use the `matchmakingUrl` value received in the decrypted payload for all Match Making API requests.
`redirectUrl`	Use the redirectUrl value received in the decrypted payload when canPlay is false

4.5 Token Implementation

To ensure proper credit validation and secure game access, the Credit Access API must be called from within the game whenever a user starts or replays a game session.

Game Launch Flow

1. User opens the Play & Win section.
2. User clicks **Play** on the GameBox platform.
3. Game launches successfully after initial validation.
4. Inside the game, when the user clicks the **Play/Start Game** button, the following API must be called:

API Endpoint

POST https://api-gamebox.stacknize.com/api/v1/user/checkCreditGameAccess

Request Payload

```
{
    "user_id": 1001,
     "game_id": 25
}
```

5. The game must validate the API response before starting gameplay.
6. Only when canPlay=true is returned should the game session begin.

Rematch Flow

When a user completes a game and selects **Rematch**:

1. User clicks the **Rematch** button inside the game.
2. Before starting the new match, the game must call the same Credit Access API again.
3. The API will revalidate the user's subscription and credit eligibility.
4. If canPlay=true, the rematch should start.
5. If canPlay=false, the game should prevent the rematch and display the message returned by the API.

Important Notes

- The Credit Access API must be called for every new game session initiated from within the game.
- The API must be called when:
    - User clicks **Play/Start Game**
    - User clicks **Rematch**

Expected Behaviour

Action	API Call Required	Result
Start Game inside Game	Yes	Validate access before gameplay
Rematch	Yes	Revalidate access before starting new match

This implementation ensures that subscription status and available credits are validated for every gameplay attempt, including rematches, maintaining consistent access control across both Multi Gaming and Play & Win environments.

Note 1: Credit Access API Authentication

The checkCreditGameAccess API requires a JWT Bearer Token for authentication.

Please use the token received in the decrypted launch payload under the bearerToken key.

Example:

Authorization: Bearer <bearerToken>

Do not use any static token. Always use the latest token received in the decrypted launch payload.

Note 2: Credit Access API Endpoint

Do not hardcode the Credit Access API URL.

Please use the API URL received in the decrypted launch payload under the checkCreditScoreAPIUrl key.

Example:

{

"checkCreditScoreAPIUrl": "https://api-gamebox.stacknize.com/api/v1/user/checkCreditGameAccess"

}

Note 3: Credit Access API Request Payload

The request payload should be constructed using the values received in the decrypted launch payload.

Use:

- userId → as user_id
- gameId → as game_id

Example Request Payload

{

"user_id": 1878931,

"game_id": 270

}

Note 4: Lobby Profile Information

In the game lobby, please display the user's profile image and display name using the values received in the decrypted launch payload.

Use the following fields:

Field in Payload	Usage
avatar_url	User Profile Image
display_name	User Display Name

Example:

{

"display_name": "User",

"avatar_url": "https://ui-gamebox.stacknize.com/assets/img/user.webp"

}

The lobby should display the user's name and profile image based on these values.

5. Nakama Connection & Session Setup

After the launch token is decrypted (or immediately, for a guest session), the game establishes a connection to the Nakama backend and authenticates the player before matchmaking can begin.

Note: For Nakama connections and related requests, use the nakamaURl value obtained from the decrypted parameters instead of a hardcoded URL.

Example:

{

"nakamaURl": "https://nakama.stacknize.com"

}

5.1 Connection Details

- **Network scheme:** supported protocols include HTTP/HTTPS REST API (http:// or https://) and persistent WebSockets (ws:// or wss://).
- **Default ports:** port 7350 (unencrypted HTTP/WS) or 443 (encrypted HTTPS/WSS).
- **Client instantiation:** the game client initializes the Nakama Client with the server details from the decrypted data.
- **WebGL builds:** use native browser fetch adapters.
- **Standalone builds:** use standard .NET HTTP clients with a 15-second network request timeout.

5.2 Authentication Flow

Authentication supports two workflows depending on the game launch context:

- **Primary flow (authenticated user):** the game extracts the platform User ID (userId) from the decrypted launch token (encryptedString URL parameter) and authenticates via Custom ID authentication.
- **Fallback flow (guest user):** if no launch token is present, the game falls back to Device ID authentication using a unique browser device identifier string.

5.3 Nakama Authentication Details

PRIMARY METHOD — CUSTOM ID AUTHENTICATION

- Method name: Client.AuthenticateCustomAsync
- HTTP endpoint: POST /v2/account/authenticate/custom
- Key used: platform User ID (userId) extracted from the decrypted encryptedString launch parameter.

FALLBACK METHOD — DEVICE ID AUTHENTICATION

- Method name: Client.AuthenticateDeviceAsync
- HTTP endpoint: POST /v2/account/authenticate/device
- Key used: device ID string (used only when the launch token is absent).

REQUEST PAYLOAD

Custom ID authentication payload (primary):

```
{
  "account_id": "1878931",
  "username": "Player_1",
  "create": true
}
```

Device ID authentication payload (guest fallback):

```
{
  "id": "e5f6g7h8-0002-0000-0000_webgl"
}
```

AUTHENTICATION API ENDPOINT

Endpoints:

- Custom ID: POST /v2/account/authenticate/custom
- Device ID: POST /v2/account/authenticate/device

HTTP headers:

- Authorization: Basic <Base64_Encoded_Server_Key> (e.g., Basic dGVzdGtleTo=)
- Content-Type: application/json

SESSION TOKEN RESPONSE FORMAT

HTTP 200 response payload:

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1aWQiOiJWV...[JWT_Token_String]",
  "created": true,
  "user_id": "1878931",
  "username": "Player_1",
  "expires_at": 1722170000
}
```

5.4 Session Creation Process

When authentication succeeds, Nakama creates and returns a user session object (ISession) containing the JWT token credentials shown in Section 5.3, which are used for all subsequent socket and API operations.

5.5 Socket Connection Creation

- **Socket instantiation:** once a valid session is created, the client instantiates a real-time WebSocket connection.
- **Handshake:** the socket connects asynchronously using the session JWT token.
- **Event binding:** upon a successful socket handshake, the client registers event listeners (e.g., ReceivedMatchmakerMatched) to handle real-time matchmaking signals.

5.6 Reconnection Handling

- **Connectivity check:** before executing any socket request (e.g., starting matchmaking or joining a match channel), an automatic connectivity check runs.
- **Transparent re-authentication:** if the WebSocket drops due to temporary network fluctuations, the client transparently re-authenticates the session and opens a fresh socket connection in the background before processing the queued request.

6. Matchmaking

When the user clicks Play, the frontend calls Nakama's matchmaking API (client.matchmake({ ... })). The game client sends a matchmaking request over the persistent Nakama socket connection, specifying the target game category, skill tier, and player level range.

Note: For Match Making API requests, use the matchmakingUrl value obtained from the decrypted parameters instead of a hardcoded URL.
Example:
{
"matchmakingUrl": "https://multiplayer.stacknize.com"
}

6.1 Matchmaking Request

Protocol: Nakama Socket API

Payload structure:

```
{
  "query": "+properties.gameType:SnakeLadder +properties.userSkill:e +properties.userLevel:>=1
+properties.userLevel:<=6",
  "minCount": 2,
  "maxCount": 2,
  "stringProperties": {
    "gameType": "SnakeLadder",
    "userSkill": "e"
  },
  "numericProperties": {
    "userLevel": 1.0
  }
}
```

6.2 Matchmaking Response

Immediately upon accepting the matchmaking request, the Nakama server returns a ticket identifier confirming that the player has entered the queue.

```
{
  "ticket": "942b0394-0f1c-4b6e-82d2-8178a9c46d29"
}
```

6.3 Bot Fill Timing

To ensure players experience fast matchmaking without long wait times, the system enforces a 30-second maximum matchmaking window:

- **0s–20s (human player search):** the platform actively searches for an online human opponent matching the skill and level criteria.
- **20s–30s (bot fill transition):** if no human opponent is matched within 20 seconds, the system automatically cancels the pending queue ticket and provisions a bot opponent in the background.
- **At 30s (guaranteed match start):** the game session begins seamlessly within 30 seconds maximum.

6.4 Error Handling

If a matchmaking attempt encounters a network interruption or socket failure, the server returns an error payload, allowing the client to notify the user gracefully:

```
{
  "error": {
    "code": 13,
    "message": "Cannot start matchmaking — socket offline or session expired.",
    "details": []
  }
}
```

6.5 SDK & Implementation Reference

SDK version: Nakama C# SDK v3.8.0 (fully compatible with Nakama Server v3.x).

Matchmaking is handled asynchronously. The reference client implementation covers the following stages, provided as platform-agnostic JavaScript/TypeScript alongside Unity code:

- Nakama client connection & session lifecycle.
- Joining & cancelling the matchmaking queue.
- Match found (matched event) & server handshake.

6.6 Match Cancellation Flow

API / EVENT

- Protocol endpoint: socket WebSocket event removeMatchmaker.
- REST API endpoint equivalent: POST /v2/matchmaker/remove.

TICKET CANCELLATION

To gracefully leave the queue, the client must send the cancellation request passing the exact ticket string identifier returned during the initial queue request (addMatchmaker). This informs the Nakama server to immediately purge the player's presence from the active matchmaking pool, preventing phantom matchups where the player is matched after they have already closed or backed out of the matchmaking screen.

Request payload example:

```
{
  "ticket": "942b0394-0f1c-4b6e-82d2-8178a9c46d29"
}
```

RESPONSE PAYLOAD

- **On success:** the server returns a success confirmation with a null/void response body, indicating the WebSocket transaction was acknowledged.
- **On failure:** if the ticket is invalid, expired, or the player has already been matched, the server returns an API exception payload:

```
{
  "error": {
    "code": 5,
    "message": "Matchmaker ticket not found or already matched.",
    "details": []
  }
}
```

7. Bot Player Details & Session Fields

When no human opponent is found within the matchmaking window (Section 6.3), the platform provisions a bot opponent. This section defines how the game identifies a bot match and which fields are provided for a bot, mirroring those provided for a human player.

7.1 Bot Identification

- **Game mode flag:** the match state transitions from PvP (Player vs. Player) to PvC (Player vs. Computer).
- **Profile attribute:** the opponent's player profile object explicitly contains the boolean property isbot: true.

7.2 Dynamic Bot Profile Selection

Rather than using static fallback values, the game dynamically queries a local profile database of pre-configured AI players.

- **Profile database:** loaded from a local structured JSON database (BotsData.json, containing 100+ distinct profiles).
- **Skill & level matching:** the system filters the database to find bots matching the player's exact skill tier (user_skill) and whose level is within +/- 5 of the player's level (user_level).
- **Randomized selection:** it selects one of the matching profiles at random to serve as the opponent, ensuring a natural and balanced match progression.

7.3 Bot Player Metadata Payload

```
{
  "userId": "bot_68249a2",
  "userName": "Computer",
  "email": "bot@stacknize.local",
  "isbot": true,
  "user_skill": "e",
  "user_level": 1,
  "avatar_url": "https://i.pravatar.cc/150?img=12",
  "role": "BOT",
  "canPlay": true
}
```

7.4 In-Game Session Context

- **Session ID:** for bot matches, the match session ID uses a local prefix (e.g., local_session_<uuid>) instead of a live Nakama match ID.
- **Opponent ID:** the opponent's identifier (stored as opponent_id in game results) matches the bot's userId (e.g., "bot_68249a2").

8. Match Connection

8.1 Connecting to the Game Server

Once a match is found, the client connects to the dedicated game server:

1. Wait for the matchmaking success response/event.
2. Read the returned matchId.
3. Connect to the multiplayer session using the received matchId.

Connection format:

```
wss://multiplayer.stacknize.com/match/993900eb-fcc0-4836-9091-e240c583fb07.nakama1
```

- Endpoint URL: wss://multiplayer.stacknize.com/match/{matchId}

- Handshake mechanism: the game client opens a standard WebSocket connection directly targeting the match path route /match/{matchId}.
- Headers / tokens / query parameters: none are appended. No custom HTTP headers (such as Authorization: Bearer <token>) and no query parameters are sent during the handshake.
- **Why it works this way:** the HTML5 WebSocket API implemented in modern web browsers does not support appending custom headers during the initial connection handshake.
- **Server-side verification:** the dedicated game server gateway validates the incoming WebSocket connection solely by matching the path parameter {matchId} and verifying that the connecting IP/client corresponds to a registered player seat for that active match session.

8.2 Match Found Event Payload

When two players are matched (or a bot is assigned), the Nakama socket broadcasts a match-found event containing the unique session ID, match join token, user presences, and server routing label:

```
{
  "ticket": "942b0394-0f1c-4b6e-82d2-8178a9c46d29",
  "match_id": "993900eb-fcc0-4836-9091-e240c583fb07.nakama1",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "label": "{\"matchPath\":\"/match/993900eb-fcc0-4836-9091-
e240c583fb07.nakama1\",\"gameType\":\"SnakeLadder\"}",
  "users": [
    {
      "presence": { "user_id": "a1b2c3d4-0001-0000-0000-000000000000", "username": "Player_1",
"session_id": "sess_111" },
      "string_properties": { "gameType": "SnakeLadder", "userSkill": "e" },
      "numeric_properties": { "userLevel": 1.0 }
    },
    {
      "presence": { "user_id": "e5f6g7h8-0002-0000-0000-000000000000", "username": "Player_2",
"session_id": "sess_222" },
      "string_properties": { "gameType": "SnakeLadder", "userSkill": "e" },
      "numeric_properties": { "userLevel": 1.0 }
    }
  ]
}
```

Nakama match channel authorization: the game client immediately passes the match token payload to the Nakama socket instance to authenticate and join the Nakama match signaling channel.

- SDK API call: Socket.JoinMatchAsync(matched), where matched contains the matching token.
- Signaling: this joins both matched players into a common Nakama chat/signaling room, allowing them to exchange basic setup messages (such as latency handshakes or custom emotes) before or during the game.

8.3 Dedicated Server Authentication

- **Dedicated server WSS:** the match token is not passed or sent to the dedicated gameplay server.

The dedicated game server verifies and authorizes incoming client connections solely based on the URL path ID (/match/{matchId}). Server instances are managed securely behind the Nakama cluster, which internally propagates player access rules based on the generated matchId when the server is provisioned — no token transmission over the WebSocket path is required from the game client.

Implementation flow:

```
[ Nakama Matchmaker ] ──(Emits Match Token)──► [ Game Client ]
        │
        ├──► 1. Join Nakama Signaling Channel: socket.JoinMatchAsync(matched)
        │
        └──► 2. Connect directly to Game Server: wss://multiplayer.stacknize.com/match/{matchId}
               (no token parameter needed)
```

8.4 Reconnect Flow

- **Detection:** the game client detects connection loss via WebSocket heartbeat failure (10-second threshold) or socket closure.
- **Reconnection HUD:** the game freezes gameplay inputs, displays a "Reconnecting..." state overlay, and enters a connection retry loop.
- **Retry attempts:** it attempts to re-establish the WebSocket connection to the exact same dedicated server URL: wss://multiplayer.stacknize.com/match/{matchId}.
- **Rejoin vs. new match:** the game always attempts to rejoin the existing active match rather than creating a new one.
- **Server-side reconnect window:** the dedicated game server reserves the disconnected player's seat for a 60-second window.

State synchronization: when the client successfully reconnects within 60 seconds, the server accepts the connection, restores the player's seat, and transmits a "State Dump" JSON message to bring the client's local view back in sync with the live server:

```
{
  "message_type": "ResumeSessionState",
  "payload": {
    "match_id": "993900eb-fcc0-4836-9091-e240c583fb07.nakama1",
    "turn_time_remaining": 14.5,
    "active_player_net_id": 5,
    "player_scores": { "player_1": 450, "player_2": 320 },
    "board_state": {
      "white_ball_pos": { "x": 1.2, "y": 0.0, "z": -2.4 },
      "remaining_pieces": [1, 2, 4, 7, 8]
    }
  }
}
```

- **Forfeit condition:** if the player fails to reconnect within the 60-second window, the server automatically forfeits the player, closes the room, and declares the remaining online player the winner.

9. Gameplay Communication

9.1 Sending Player Input

- **Connection protocol:** player inputs are sent over a persistent Secure WebSocket (WSS) connection directly to the active dedicated game server URL obtained after matchmaking (wss://multiplayer.stacknize.com/match/{match_id}).
- **Transmission mode:** inputs are transmitted in real time as asynchronous binary or JSON frames the instant a player performs an action in the browser (e.g., releasing a cue stick, rolling dice, or dropping a piece).
- **Server handshake:** messages travel across the existing WebSocket channel established during match connection, avoiding HTTP overhead and enabling low-latency gameplay.

INPUT EVENT PAYLOAD FIELDS

- message_type: string header identifying the action (e.g., "TAKE_SHOT", "ROLL_DICE").
- player_id: unique identifier of the player taking the turn.
- input_data: object containing action-specific physics or gameplay parameters (e.g., force vectors, shot angle, or spin).

Example A — aim & shot input event (Pool / Carrom):

```
{
  "message_type": "TakeShot",
  "player_id": "1878931",
  "parameters": {
```

```
    "cue_force": { "x": 0.0, "y": 0.0, "z": 18.5 },
    "shot_angle": 42.5,
    "spin_vector": { "x": 0.1, "y": 0.0, "z": -0.2 }
  }
}
```

Example B — dice roll event (Snake & Ladder):

```
{
  "message_type": "RollDice",
  "player_id": "1878931",
  "player_index": 0
}
```

9.2 Receiving Game State Updates

The dedicated game server operates authoritatively, executing physics and turn logic. It continuously broadcasts updated state payloads to all connected participants:

```
{
  "message_type": "TurnStateUpdate",
  "payload": {
    "active_player_net_id": 5,
    "turn_time_remaining": 30.0,
    "current_game_state": "WAITING_FOR_INPUT",
    "player_1_score": 450,
    "player_2_score": 320,
    "is_fault": false
  }
}
```

10. Match End

10.1 Opponent Disconnect Flow

When an opponent disconnects or leaves during the match, and fails to reconnect within the 60-second grace window (Section 8.4):

Event received: the authoritative dedicated server broadcasts a MatchEndNotification message over the WebSocket to the remaining online player.

```
{
  "message_type": "MatchEndNotification",
  "winner_user_id": "1878931",
  "game_end_reason": "forfeit",
  "final_scores": { "player_1": 1500, "player_2": 0 }
}
```

- **Winner declared automatically:** the remaining connected player is automatically declared the winner. Their status is set to "is_winner": true, and the opponent is marked as defeated.
- **Score submitted immediately:** the client immediately triggers the score submission API call (submitMultiplayerScore) upon receiving the forfeit notice, submitting a payload containing "is_winner": true, the current score, and the common stat "game_end_reason": "forfeit", so the backend immediately saves the match outcome and settles the tournament.

10.2 Normal Match Completion

The dedicated game server authoritatively monitors and decides when a match ends (e.g., final ball potted, tile 100 reached, or board cleared). Upon game over, the server sends a match termination message to both clients — clients do not dictate match completion to the game server.

Upon receiving the match termination message from the game server, the WebGL game client executes an HTTP POST request to submit match statistics to the platform API:

- API endpoint: POST https://test-gamebox.stacknize.com/api/api/v1/tournaments/submitMultiplayerScore
- Header: Content-Type: application/json

10.3 Match End Event Payload

When a match completes normally (or terminates due to a forfeit), the dedicated server broadcasts a MatchEndNotification JSON payload over the WebSocket to notify the clients of the final results:

```
{
  "message_type": "MatchEndNotification",
  "winner_user_id": "1878931",
  "game_end_reason": "completed",
  "final_scores": { "player_1_user_id": 1500, "player_2_user_id": 820 },
  "match_summary": { "total_turns": 14, "duration_seconds": 240 }
}
```

10.4 Client Reaction on Receipt

1. Freeze gameplay — the client locks the user interface and displays the win/loss screen based on winner_user_id.
2. Score API call — the client immediately packages the final scores/statistics, encrypts them using the scoreEncryptionKey, and POSTs the payload to the platform's submitMultiplayerScore API.

11. Server Error Events

11.1 Invalid Token / Unauthorized

Trigger: occurs during the session authentication request (POST /v2/account/authenticate/custom) if the platform launch token is malformed, has expired, or fails HMAC signature validation.

HTTP status code: 401 Unauthorized or 403 Forbidden.

```
{
  "error": {
    "code": 16,
    "message": "Auth token is invalid, expired, or has been tampered with.",
    "details": []
  }
}
```

Game behavior: the game client aborts the loading phase, fires a connection failure event, resets user profile information, and falls back to Device ID (guest) login.

11.2 Room Closed

Trigger: occurs when the client attempts to connect to wss://multiplayer.stacknize.com/match/{matchId} but the game server instance has already terminated or the match channel is closed.

- WebSocket close code: 1006 Abrupt Closure or 1008 Policy Violation.

Game behavior: the WebSocket connection handler catches the socket close event, aborts the game scene transition, resets the UI, and returns the player to the lobby with the message "Connection to game server was lost or the room has closed."

11.3 Match Expired

Trigger: occurs when the client attempts to join a matchmaking session via Nakama (JoinMatchAsync) using a matched token, but the token's TTL (time-to-live) has elapsed.

```
{
  "error": {
    "code": 5,
    "message": "Match token expired or session not found.",
    "details": []
  }
}
```

Game behavior: the client cancels the active matchmaking ticket, logs the timeout exception, and triggers UI panels to return the player to the matchmaking lobby screen to request a new ticket.

11.4 Server Unavailable

Trigger: occurs if the Nakama server is offline, network connectivity is lost, or the dedicated server host is unreachable.

- Network error: TCP Timeout / Connection Refused.

Game behavior: the connection manager enforces a 15-second REST timeout and a 5-second socket timeout. When these expire without an active handshake response, the client cancels the operation, tears down the socket, and alerts the user: "Server is temporarily unavailable. Please check your network connection."

12. Score Encryption

Once the match concludes, the game must encrypt the score payload using the scoreEncryptionKey obtained during launch, then submit it to the platform.

12.1 Sample Score Payload

```
{
  "userId": "1878931",
  "gameId": "229",
  "session_id": "123165465",
  "is_winner": true,
  "score": 1324688,
  "stats": {}
}
```

12.2 Encryption Process

1. Decrypt the launch data (see Section 4).
2. Read the scoreEncryptionKey.
3. Read the gameScoreAPIUrl.
4. Create the multiplayer score payload containing user, game, session, score, and statistics data.
5. Validate the required fields: userId, gameId, and session_id.
6. Convert the payload to a JSON string.
7. Compress the JSON string using zlib DeflateRaw.
8. Generate the AES key from scoreEncryptionKey.
9. Generate the IV from scoreEncryptionKey.
10. Encrypt the compressed payload using AES-128-CBC.
11. Generate an HMAC-SHA256 signature for the encrypted data.

12. Base64URL-encode the encrypted ciphertext.
13. Base64URL-encode the generated HMAC signature.
14. Create the token structure containing v, ct, and mac.
15. Convert the token structure to a JSON string.
16. Base64URL-encode the final token JSON to produce the encryptedString.
17. Send the resulting encryptedString in the submitMultiplayerScore API request.

12.3 Token Structure

Both the launch token and the score submission token use the same structure:

```
{
  "v": 1,
  "ct": "<ciphertext>",
  "mac": "<hmac>"
}
```

13. Algorithms Used

- AES-128-CBC — symmetric encryption of launch and score payloads
- HMAC-SHA256 — integrity and authenticity verification
- Base64URL Encoding — URL-safe transport encoding for ciphertext and signatures
- zlib DeflateRaw — payload compression prior to encryption
- zlib InflateRaw — payload decompression after decryption

14. Compatibility Requirement

Vendors must use the same algorithms, token structure, compression format, and encoding scheme described in this document. Any deviation will cause MAC validation or decryption to fail on the platform side.

15. Score Submission Flow

7. Generate the encryptedString as described in Section 12.
8. Call the gameScoreAPIUrl endpoint.
9. Submit the payload containing the encryptedString.
10. submitMultiplayerScore validates the token.
11. submitMultiplayerScore decrypts the token.
12. The score is saved.
13. Statistics are saved.
14. Winner status is saved.

16. Multiplayer Game Score Submission API

16.1 Endpoint

```
POST  https://test-gamebox.stacknize.com/api/api/v1/tournaments/submitMultiplayerScore
```

16.2 Request Payload

```
{
  "encryptedString": "<generated encrypted string>"
}
```

16.3 cURL Example

```
curl --location 'https://test-
gamebox.stacknize.com/api/api/v1/tournaments/submitMultiplayerScore' \
  --header 'Content-Type: application/json' \
  --data '{"encryptedString": "<generated encrypted string>"}'
```

16.4 Response

```
{
  "message": "Success !!"
}
```

16.5 Functionality

- Saves the player score.
- Saves winner status.
- Saves common game statistics.
- Saves game-specific statistics.
- Supports multiple players within the same session.
- Handles missing statistics gracefully.

__Note:__ Sample values shown throughout this document (keys, tokens, IDs, session/match identifiers) are drawn from the staging/test environment and are provided for illustration only. Production credentials will be issued separately through a secure channel. Please provide the __game orientation (Portrait/Landscape)__ and the __required dimensions/resolution (Width × Height)__ for each game. This information is required to ensure that every game is displayed correctly across the platform and devices.